SkillPilot

FREE LEARNING ROADMAP

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score

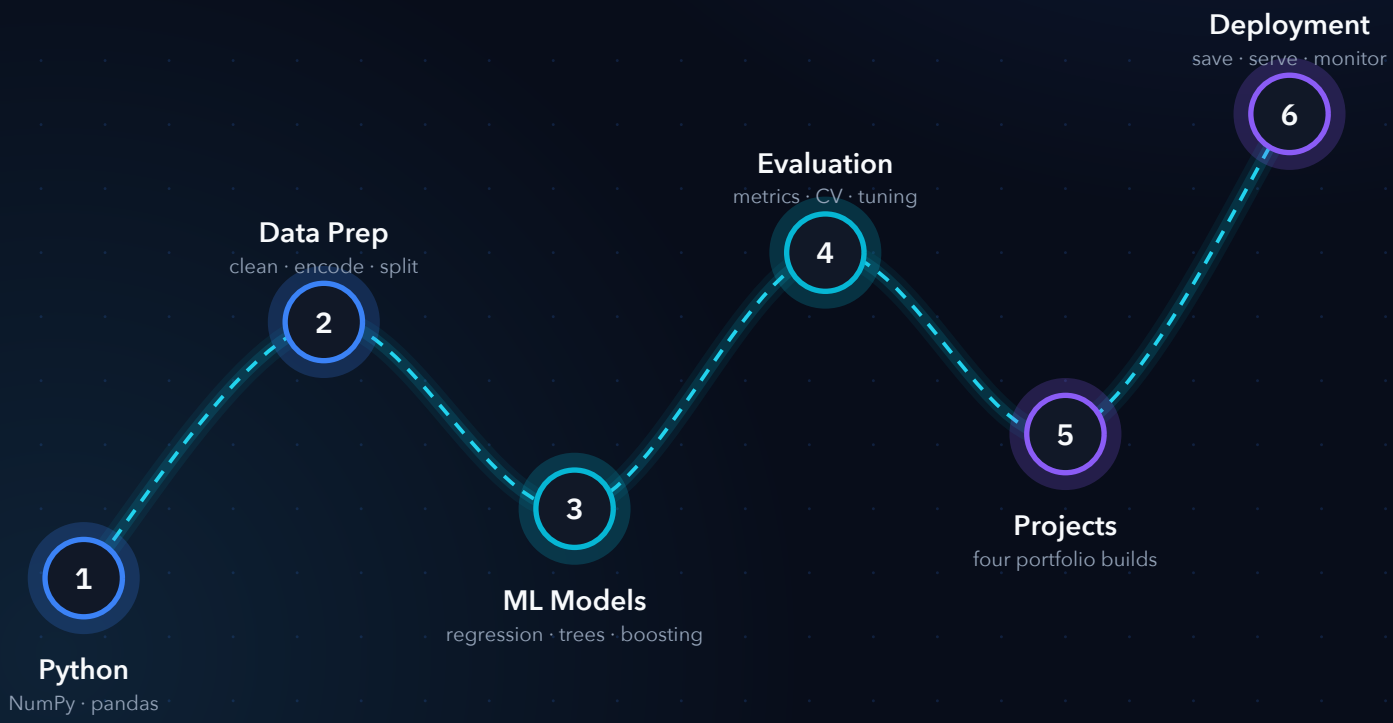
model = Pipeline([
    ("prep", preprocess),
    ("clf", HistGradientBoostingClassifier()),
])
scores = cross_val_score(model, X, y, cv=5,
                         scoring="average_precision")
print(scores.mean(), scores.std())
```

THE

MACHINE LEARNING ROADMAP_

From Python Fundamentals to Real-World ML Projects

Learn in sequence. Practice early. Build real projects.



9
learning stages

4
portfolio projects

12-week
sample study plan

NAVIGATE YOUR SKILLS. BUILD YOUR FUTURE.

START HERE

How to Use This Roadmap

This roadmap answers the questions most beginners get stuck on: what to learn first, how much math you need, which algorithms matter, how to practice, which projects to build, and what comes after the fundamentals. It is written for aspiring data scientists and ML engineers who are willing to learn Python, and it assumes no prior machine learning.

01 **Follow the stages in order**
Each stage uses the one before it. Python lets you handle data, statistics helps you understand it, preprocessing makes it usable, and only then do models, evaluation and deployment make sense. Skipping prerequisites is the most common reason learners feel lost in the middle of a course.

02 **Start projects early**
You do not need every algorithm before you build something. Begin Project 1 while you are still in Stages 3 and 4, and let each project pull you into the next topic. A small project that works teaches more than three finished courses.

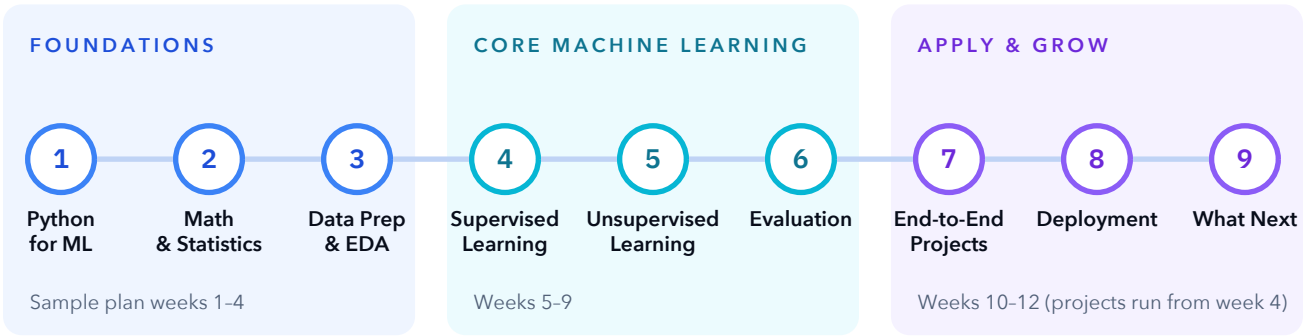
03 **Use the practice task and checkpoint**
Every stage ends with a **practice task** and a **readiness checkpoint**. Move on when you can do most of the checkpoint without notes. If you cannot, spend another few days on practice rather than on new videos.

04 **Track progress with the checklists**
The progress tracker and final readiness checklist near the end have empty boxes to tick on paper or in a PDF annotation tool. Date each milestone so you can see your pace.

05 **Avoid tutorial hopping**
Pick **one** main resource per stage and use official documentation as a reference. Watching or reading without typing code feels productive but rarely sticks. Aim to spend at least half of your study time writing code.

> **A SEQUENCE, NOT A RULEBOOK**
> This roadmap is a practical sequence, not a rulebook. Stages overlap, and you will revisit earlier ones as you go. People learn ML successfully in other orders; what matters is steady practice and real projects.

The journey in brief

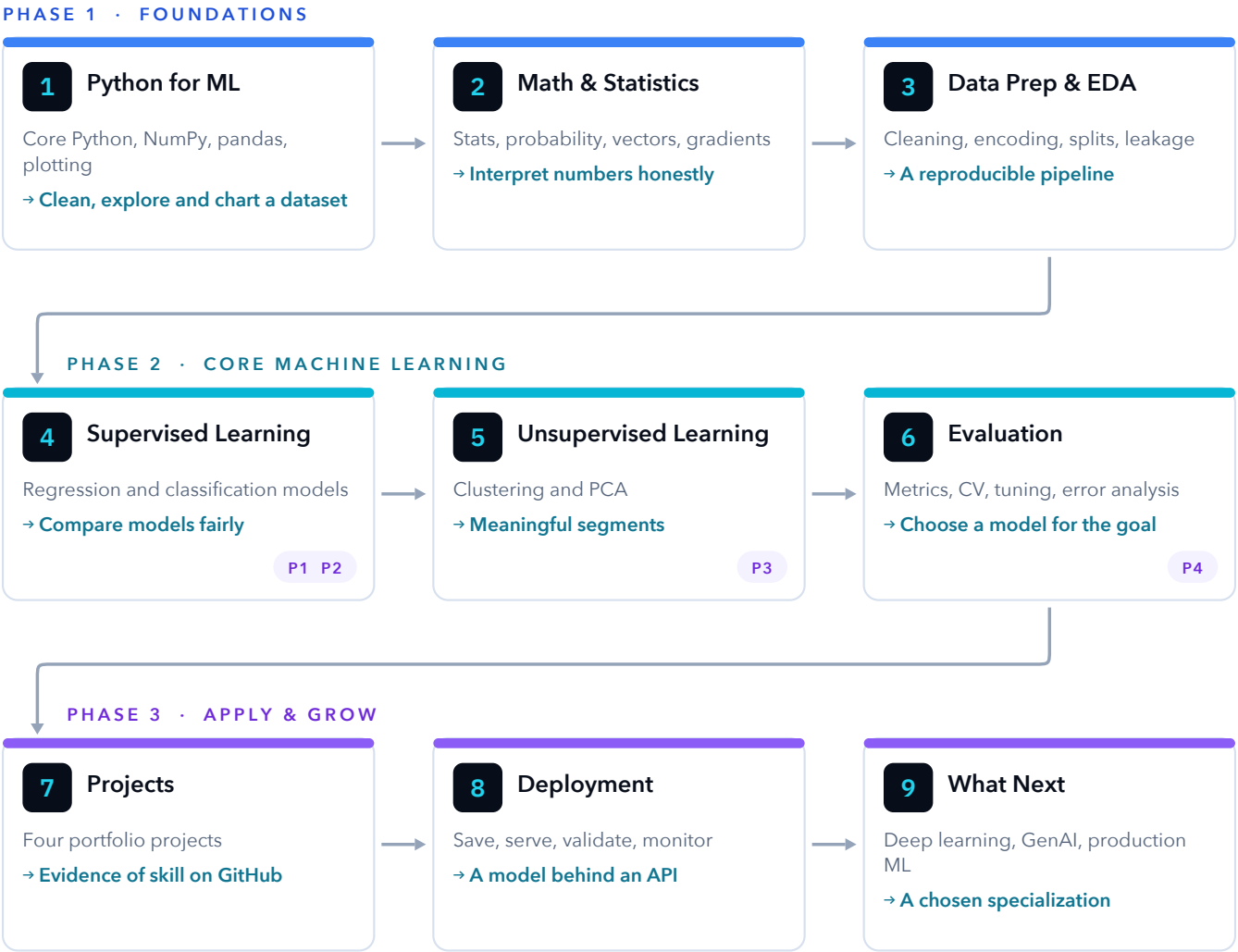


CONTENTS

The Roadmap at a Glance

- How to Use This Roadmap 02
- The Roadmap at a Glance 03
01 Python for Machine Learning 04
02 Mathematics and Statistics 05
03 Data Preprocessing and EDA 07
04 Supervised Machine Learning 09
05 Unsupervised Learning 11
06 Model Evaluation and Improvement 13
07 End-to-End Projects 15
08 Deployment and Production Basics 18
09 What to Learn Next 20
- 12-Week Learning Plan 21
- Progress Tracker and Readiness Checklist 22
- Learning Resources 23

Full roadmap flowchart



P1-P4 Where each portfolio project starts (Stage 7 describes them). Evaluation often sends you back to the data, so expect to loop between Stages 3 and 6 several times.

STAGE 01 FOUNDATIONS

Python for Machine Learning

01

Write small Python programs and work with tabular data without copying every line.

Every later stage is written in Python. You do not need to be a software engineer, but you need enough fluency that the code stops being the hard part, so your attention can go to the data and the model.

Core Python: what you should be able to do

TOPIC	WHAT YOU SHOULD BE ABLE TO DO
Syntax and control flow	Use variables, `if`/`elif`, `for` and `while` loops and `range` without looking them up.
Functions	Write functions with parameters, defaults and return values, and reuse them.
Lists, tuples, dicts, sets	A list for order, a dict for lookup by key, a set for uniqueness, a tuple for a fixed record.
Comprehensions	Write `[x * 2 for x in values if x > 0]` and dict comprehensions, and know when a plain loop is clearer.
Exception handling	Read a traceback, catch specific errors with `try`/`except`, and raise a clear error for bad input.
Modules and packages	Import from the standard library and from your own `.py` files, not one giant notebook.
File handling	Open files with `with open(...)`, read and write text, and build paths with `pathlib`.
CSV and JSON	Read and write both formats with the `csv` and `json` modules, then with pandas.
Basic OOP	Define a class with `__init__` and methods; understand why scikit-learn models expose `fit` and `predict`.
Virtual environments	Create one with `python -m venv .venv`, install with `pip`, and record versions in `requirements.txt`.
Debugging	Use `print`, `assert` and a debugger breakpoint to find where a value goes wrong.
Readable code	Clear names, short functions, comments that explain why, and consistent formatting.

The four data libraries

LIBRARY	WHAT IT IS USED FOR	YOU SHOULD BE ABLE TO
NumPy	Fast n-dimensional arrays and vectorized math. pandas and scikit-learn are built on it.	Create, index and slice arrays; reason about `shape` and `axis`; replace loops with vectorized operations.
pandas	Tabular data in DataFrames: loading, cleaning, filtering, joining and grouping.	Read a CSV, inspect it with `info()` and `describe()`, filter rows, handle missing values, `groupby` and `merge`.
Matplotlib	The base plotting library, with full control over figures and axes.	Draw line, bar, scatter and histogram plots, label axes and titles, and save a figure.
Seaborn	Statistical charts built on Matplotlib with less code.	Plot distributions, box plots, count plots and correlation heatmaps directly from a DataFrame.

PRACTICE TASK

Choose a public tabular dataset with a few thousand rows and mixed column types. In one notebook: load it with pandas; inspect `shape`, `dtypes` and missing values; fix three concrete issues (for example wrong types, inconsistent category labels, duplicate rows); compute summary statistics by group; and draw four labeled charts that each answer one question. Finish with five plain-English findings.

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] You can write a 50 to 100 line script from a blank file, using functions, without a tutorial open.
- [ ] You can load, filter, group and aggregate a CSV in pandas, looking up only unusual methods.
- [ ] You can read a traceback and find the failing line yourself.
- [ ] Your project runs in its own virtual environment from a `requirements.txt` file.

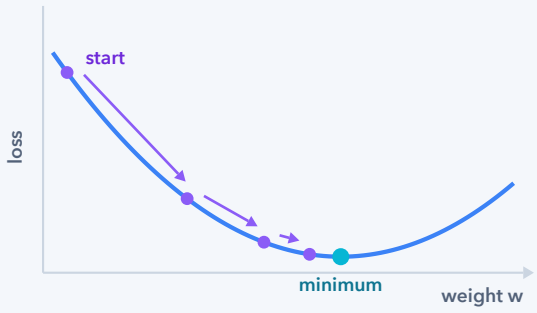
STAGE 02 FOUNDATIONS

Mathematics and Statistics

02

Build intuition for the numbers behind models.
Intuition first, proofs later.

Machine learning is applied statistics and optimization, written in the language of linear algebra. To start, you need to understand what the numbers mean, not to prove theorems. Study the **Essential now** column alongside Stages 3 and 4, and return to **Deepen later** when a model or role needs it.



Gradient descent in one picture. You can train your first model before finishing this stage. Many learners understand gradient descent best after watching a model's loss fall in their own code.

Update rule: `w_new = w - learning_rate * slope_of_loss(w)`. A learning rate that is too large overshoots the minimum; one that is too small crawls.

Statistics and probability

Describe data, reason about uncertainty, avoid false conclusions.

ESSENTIAL NOW	WHY IT MATTERS IN ML
Mean, median, mode, variance, standard deviation	Summarize a column, spot skew (mean far from median), compare groups.
Probability and conditional probability	P(A | B) is the language of classifiers and the basis of Naive Bayes.
Common distributions	Normal, binomial, uniform, Poisson; recognize skewed and heavy-tailed data.
Sampling and sampling bias	Does your data represent the people or events the model will meet later?
Correlation versus causation	Correlation measures association. It cannot tell you what causes what.

ESSENTIAL NOW	WHY IT MATTERS IN ML
Confidence intervals, hypothesis tests (intro)	How sure can you be that a difference is real, for example in an A/B test?
DEEPEN LATER	Maximum likelihood, Bayesian inference, statistical power, multiple-testing correction.

Linear algebra

The notation every ML library and paper uses.

ESSENTIAL NOW	WHY IT MATTERS IN ML
Scalars, vectors and matrices	A dataset is a matrix: rows are examples, columns are features.
Matrix operations and shapes	Addition, transpose, and why `(100, 3) @ (3, 1)` works but `(100, 3) @ (100, 1)` fails.
Dot product	A linear model's prediction is a dot product of weights and features, plus a bias.
Matrix multiplication	How a whole batch of predictions is computed in one step.
Vector norms (L1, L2)	Distances in kNN and k-means; the penalties in Lasso and Ridge.
Eigenvalues and eigenvectors (intuition)	The directions of greatest variance that PCA finds (Stage 5).
DEEPEN LATER	Matrix inverse and rank, SVD and other decompositions, matrix calculus.

Calculus and optimization

How models learn: by reducing a loss step by step.

ESSENTIAL NOW	WHY IT MATTERS IN ML
Functions and graphs	Read a curve: where it rises, falls and reaches a minimum.
Derivatives	The slope of a function: how much the output changes for a small input change.
Partial derivatives and gradients	The gradient collects partial derivatives; it points in the direction of steepest increase.
Gradient descent	Repeatedly step opposite to the gradient. The learning rate sets the step size.
Loss functions	Mean squared error for regression, log loss for classification: the number training minimizes.
DEEPEN LATER	Chain rule in depth (needed for backpropagation), convexity, optimizers such as Adam.

PRACTICE TASK

Take a small dataset (a few hundred rows of house sales or exam scores). Compute mean, median and standard deviation per column and use mean versus median to detect skew. Build a correlation matrix and pick the two strongest relationships; for each, write what it suggests and one reason it might not be causal (a confounder, reversed direction or sampling bias). Then implement gradient descent for one-feature linear regression in NumPy and plot the loss for each iteration.

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] You can explain variance and standard deviation using a column from your own data.
- [ ] Given a correlation of 0.8, you can name two reasons it may not mean causation.
- [ ] You can compute a dot product by hand and explain how a linear model uses it.
- [ ] You can describe what gradient descent does and what happens when the learning rate is too large.

STAGE 03 FOUNDATIONS

Data Preprocessing and EDA

03

Turn raw, messy data into a clean, honest model input that can be rebuilt with one command.

Most of the work in a real ML project happens before training. Choices made here often change results more than the choice of algorithm. Two terms you will use constantly: a **feature** is an input column used to make a prediction, and the **target** (or label) is the value you want to predict. **Exploratory data analysis (EDA)** means investigating the data with summaries and charts before modeling it.

Topics, their purpose, and the classic mistake

TOPIC	PURPOSE	COMMON MISTAKE
Feature types	Numeric, categorical (nominal or ordinal), datetime, text and ID columns each need different handling.	Treating ZIP codes or customer IDs as numbers the model can do arithmetic on.
Data cleaning	Fix types and units; unify labels such as “UK”, “U.K.” and “United Kingdom”.	Dropping rows silently, without recording how many and why.
Missing values	Drop, impute (median, most frequent) or add an indicator column; missingness can itself be informative.	Computing imputation values on the full dataset, including test rows.
Duplicate records	Remove exact duplicates; investigate near-duplicates.	Copies of the same record landing in both train and test, inflating scores.
Outlier investigation	Decide whether an extreme value is an error, rare but real, or exactly what you want to detect.	Deleting everything beyond 3 standard deviations by reflex.
Categorical encoding	One-hot encoding for nominal categories; ordinal encoding only when order is real.	Ignoring categories that first appear at prediction time.
Feature scaling	Standardize features for distance- and gradient-based models: kNN, SVM, regularized linear models, k-means, PCA.	Fitting the scaler before splitting. (Trees do not need scaling.)
Train / validation / test	Train to fit, validate to choose, test once at the end to estimate performance on unseen data.	Checking the test set repeatedly; random splits on time-ordered data.
Exploratory data analysis	Distributions, relationships with the target, missingness patterns and data-quality questions.	Plotting everything without a question in mind, or skipping EDA.
Feature engineering	Create informative inputs: ratios, date parts, counts, per-customer aggregates.	Using information that will not exist at prediction time.

TOPIC	PURPOSE	COMMON MISTAKE
Class imbalance	Notice when one class is rare; use stratified splits, class weights, suitable metrics and thresholds.	Oversampling before the split, so copies leak into validation.
Data leakage	Information from outside the training data, or from the future, reaching the model during training.	A `refund_issued` feature in a fraud model meant to run at checkout.
Reproducible pipelines	Chain preprocessing and model in a scikit-learn `Pipeline` so identical steps run in training and production.	Manual notebook cells that only work when run in one particular order.

A leakage-safe preprocessing pipeline in scikit-learn

pipeline.py SCIKIT-LEARN

```python
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric = ["tenure_months", "monthly_charges"]
categorical = ["contract", "payment_method"]

preprocess = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numeric),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), categorical),
])
model = Pipeline([("prep", preprocess), ("clf", LogisticRegression(max_iter=1000))])

# Split BEFORE fitting anything, so test rows never influence preprocessing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
model.fit(X_train, y_train)   # imputers and scaler learn from training rows only
print(model.score(X_test, y_test))
```

`ColumnTransformer` applies different steps to numeric and categorical columns. Because everything sits inside one `Pipeline`, cross-validation and the final API repeat exactly the same preprocessing, fitted on training rows only. `X` and `y` are your feature DataFrame and target.

PRACTICE TASK

Use a customer churn-style dataset (search: “telco customer churn dataset”). Define the target and the moment when a prediction would be made. Audit types, missing values, duplicates and outliers; engineer at least three features (for example charges per month of tenure, or number of add-on services); split with stratification before fitting anything; and build a `ColumnTransformer` + `Pipeline`. Keep a decision log: each change, why you made it, and how many rows or columns it affected.

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] For every column you can state its type and how you will process it.
- [ ] You can explain data leakage with an example from your own project.
- [ ] Your data goes from raw file to model input through one pipeline, with no manual steps.

STAGE 04 CORE MACHINE LEARNING

Supervised Machine Learning

04

Understand how the main tabular algorithms work and when to reach for each one.

In supervised learning every training example comes with a known answer, the label. The model learns a mapping from features to that label. **Regression** predicts a number (price, demand). **Classification** predicts a category (churn or no churn). You do not need every algorithm before your first project: learn the **Core** models first, and treat the rest as tools to add when a problem calls for them.

CORE: LEARN FIRST	Linear regression · Logistic regression · Decision trees · Random forests · Gradient boosting
KNOW WELL	Ridge and Lasso · k-Nearest Neighbors · Naive Bayes
SITUATIONAL	Support Vector Machines · Polynomial features

Regression

Linear Regression — Predicting a continuous number from features. CORE

HOW IT WORKS	Fits a weighted sum of features plus an intercept, choosing weights that minimize mean squared error.
USEFUL WHEN	As an interpretable first model, and when the relationship is roughly linear.
WATCH OUT	Sensitive to outliers; correlated features make weights unstable; weights describe association, not cause.
PRACTICE	Fit on housing data, interpret the coefficients, and plot residuals against predictions.

Ridge and Lasso — Overfitting and unstable weights in linear models. KNOW WELL

HOW IT WORKS	Add a penalty on weight size to the loss. Ridge (L2) shrinks all weights; Lasso (L1) can set some exactly to zero.
USEFUL WHEN	Many or correlated features; Lasso when you also want automatic feature selection.
WATCH OUT	Features must be scaled; the penalty strength `alpha` must be tuned with cross-validation.
PRACTICE	Compare plain, Ridge and Lasso across `alpha` values and note which weights Lasso removes.

Polynomial features — Curved relationships, using a linear model. SITUATIONAL

HOW IT WORKS	Adds squared and interaction terms, then fits an ordinary linear model on the expanded features.
USEFUL WHEN	A few features with a clear, smooth curve.
WATCH OUT	Feature count grows fast with degree; high degrees overfit and extrapolate wildly.
PRACTICE	Fit degrees 1, 2, 5 and 10 and compare training error with validation error.

Classification

Logistic Regression
Binary or multiclass classification with probability outputs. **CORE**

HOW IT WORKS	Passes a linear score through the sigmoid function to get a probability between 0 and 1; trained by minimizing log loss.
USEFUL WHEN	A fast, interpretable baseline that is surprisingly hard to beat on many problems.
WATCH OUT	Linear decision boundary unless you engineer features; scale inputs; regularized by default (`C`).
PRACTICE	Build a churn baseline, inspect coefficients, and see how results change with the threshold.

k-Nearest Neighbors
Classification or regression by similarity. **KNOW WELL**

HOW IT WORKS	Predicts from the k closest training examples, measured by distance.
USEFUL WHEN	Small datasets; building intuition for distance and scaling.
WATCH OUT	Needs scaling; slow to predict on large data; weakens as feature count grows.
PRACTICE	Plot validation score against k, with and without scaling.

Naive Bayes
Very fast classification, especially of text. **KNOW WELL**

HOW IT WORKS	Applies Bayes' theorem while assuming features are independent given the class.
USEFUL WHEN	Spam and text classification; quick baselines on small data.
WATCH OUT	Independence rarely holds; probabilities are often poorly calibrated even when rankings are good.
PRACTICE	Build a spam classifier with word counts and `MultinomialNB`.

Decision Trees
Classification or regression with readable if-then rules. **CORE**

HOW IT WORKS	Repeatedly splits the data on the feature and threshold that best separate the target values.
USEFUL WHEN	When you need to explain logic; mixed feature types; no scaling needed.
WATCH OUT	Deep trees overfit; small data changes can produce a very different tree.
PRACTICE	Vary `max_depth`, plot the tree, and compare training and validation scores.

Random Forests
Robust prediction on tabular data. **CORE**

HOW IT WORKS	Trains many trees on bootstrap samples with random feature subsets, then averages their predictions.
USEFUL WHEN	A strong default that needs little tuning.
WATCH OUT	Larger and less interpretable; built-in importances can mislead, so prefer permutation importance.
PRACTICE	Compare with a single tree; compute permutation importance on validation data.

Gradient Boosting
High-accuracy prediction on structured, tabular data. **CORE**

HOW IT WORKS	Builds trees one after another, each correcting the errors of the ensemble so far.
USEFUL WHEN	Often among the strongest performers on tabular problems. XGBoost, LightGBM and CatBoost are popular libraries; learn one after scikit-learn's version.
WATCH OUT	More hyperparameters; overfits without early stopping; balance learning rate against number of trees.
PRACTICE	Use `HistGradientBoostingClassifier` with early stopping, then try one external library.

Support Vector Machines

Classification (and regression) with a maximum-margin boundary. SITUATIONAL

HOW IT WORKS	Finds the boundary with the widest margin between classes; kernels allow curved boundaries.
USEFUL WHEN	Small to medium datasets with many features, such as text.
WATCH OUT	Needs scaling; kernel SVMs scale poorly to large data; `C` and `gamma` need tuning.
PRACTICE	Compare linear and RBF kernels; tune `C` and `gamma` with a grid search.

PRACTICE TASK

On one classification dataset, compare a `DummyClassifier` baseline, logistic regression, a random forest and gradient boosting, all inside the same pipeline and the same cross-validation folds. Report a table with the mean and spread of your metric and the training time, plus two sentences per model explaining why it did better or worse.

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] You can explain, without notes, how linear regression, logistic regression, a tree and a random forest predict.
- [ ] You can pick and justify a baseline and two candidate models for a new tabular problem.
- [ ] You can explain why kNN, SVMs and regularized linear models need scaling but trees do not.

STAGE 05 CORE MACHINE LEARNING

Unsupervised Learning

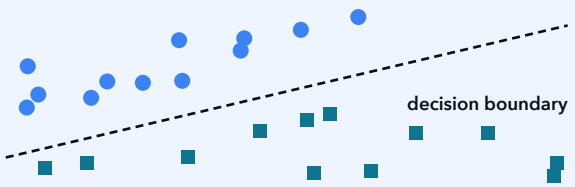
05

Find structure in unlabeled data, and judge whether that structure is useful.

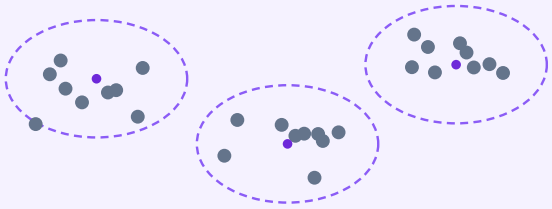
Unsupervised learning looks for patterns without a label to predict. Because there is no correct answer to compare against, evaluation relies on internal measures, stability checks and domain knowledge rather than a single score. Typical uses are customer segmentation, grouping similar documents or products, spotting unusual observations, and exploring a dataset before supervised modeling.

Supervised versus unsupervised learning

SUPERVISED labels known: learn the boundary



UNSUPERVISED no labels: discover the groups



DATA	Labeled examples: features plus a known answer	Unlabeled examples: features only
GOAL	Learn a mapping from features to the label	Find groups, structure or compact representations
EVALUATION	Compare with true labels on held-out data	Internal scores, stability and usefulness
EXAMPLES	Predict house prices; flag likely churners	Segment customers; compress 50 features to 2

Core methods

METHOD	HOW IT WORKS	USEFUL WHEN	WATCH OUT
k-Means Clustering	Assigns each point to the nearest of k centers, moves each center to the mean of its points, and repeats until stable.	Compact, similar-sized groups; customer segmentation.	You choose k (elbow plot, silhouette score, business sense); scale features; sensitive to outliers; use several initializations.
Hierarchical clustering Clustering	Agglomerative version: starts with every point as its own cluster and repeatedly merges the closest two, producing a tree (dendrogram).	Small to medium datasets; seeing structure at several levels of detail.	Time and memory grow quickly with rows; the linkage choice changes results.
DBSCAN (introductory) Clustering	Groups points in dense regions; points in sparse regions are labeled noise.	Irregularly shaped clusters; finding outliers; no need to choose k.	Sensitive to `eps` and `min_samples`; struggles when clusters differ in density.
Principal Component Analysis Dimensionality reduction	Finds new axes (principal components) that capture the most variance, and keeps the first few.	Plotting high-dimensional data in 2D; reducing many correlated features.	Linear only; components mix original features, so they are harder to interpret; scale first; check explained variance.

Feature representation and visualization. Clusters are only as good as the features you give them: choose and transform features (for example log of spending) to reflect the similarity you care about. Plot clusters on the first two principal components, and describe each cluster with a profile table of medians. Methods such as t-SNE and UMAP make attractive 2D plots, but distances in those plots are easy to misread; treat them as later, visualization-only topics.

PRACTICE TASK

Build recency, frequency and monetary (RFM) features per customer from transaction data (search: “online retail transactions dataset UCI”). Scale them, run k-means for k = 2 to 8, and compare silhouette scores and cluster sizes. Profile each cluster with medians, give it a business name, and re-run with different random seeds and a sample of the data. Are the segments stable, distinct and actionable?

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] You can explain the difference between supervised and unsupervised learning with one example of each.
- [ ] You can justify a choice of k with at least two kinds of evidence.
- [ ] You can plot data on two principal components and explain what the explained-variance ratio means.

STAGE 06 CORE MACHINE LEARNING

Model Evaluation and Improvement

Measure how well a model will generalize, and improve it without fooling yourself.

06

Fitting a model means adjusting its parameters to the training data. **Evaluating** it means estimating how well it will perform on new data it has never seen, which is called **generalization**. A model can fit its training data perfectly and still fail in production. Everything in this stage exists to estimate generalization honestly.

Core evaluation concepts

CONCEPT	WHAT TO KNOW
Baseline models	A trivial predictor such as the mean value or the most frequent class (`DummyRegressor`, `DummyClassifier`). Any real model must beat it.
Train, validation, test	Fit on training data, compare and tune on validation data, and touch the test set once at the very end.
Cross-validation	k-fold CV trains and scores k times on different splits, giving a mean and a spread. Use stratified folds for classification and time-based splits for time-ordered data.
Overfitting and underfitting	Overfitting: excellent training score, much worse validation score (memorizing). Underfitting: poor on both (too simple). Learning curves show which one you have.
Bias-variance trade-off	Simple models make systematic errors (high bias); very flexible models are sensitive to the particular sample (high variance). Regularization, more data and ensembles move the balance.
Hyperparameter tuning	Settings chosen before training, such as tree depth, `C` or `alpha`. Search with `GridSearchCV` or `RandomizedSearchCV` inside cross-validation on training data only.
Threshold selection	Classifiers output scores; the default 0.5 cut-off is rarely the best. Choose the threshold on validation data to match the cost of each error type (`TunedThresholdClassifierCV`).
Error analysis	Read the worst predictions and group errors by segment. It often reveals data problems and new feature ideas faster than more tuning.
Feature importance	Coefficients, permutation importance and partial dependence plots explain what drives predictions. Importance describes the model, not causes in the real world.
Reproducibility	Fix random seeds, version code and data, record library versions, and keep all preprocessing inside the pipeline.

Metrics comparison

METRIC	TASK	WHAT IT MEASURES	USE WHEN	CAUTION
MAE	Regression	Average absolute error, in target units	Error cost grows in proportion to its size; easy to explain	Does not emphasize rare large errors
RMSE	Regression	Square root of mean squared error	Large errors are especially costly	Sensitive to outliers

METRIC	TASK	WHAT IT MEASURES	USE WHEN	CAUTION
R^2	Regression	Share of variance explained versus predicting the mean	Comparing models on the same data	Not in target units; can be negative on new data
Accuracy	Classification	Share of all predictions that are correct	Balanced classes, similar error costs	Misleading when one class is rare
Precision	Classification	Of predicted positives, the share that are truly positive	False alarms are costly (e.g. blocking real emails as spam)	Ignores positives the model missed
Recall	Classification	Of actual positives, the share the model found	Misses are costly (fraud, disease screening)	Ignores false alarms
F1 score	Classification	Harmonic mean of precision and recall	Need a balance of both; positive class is rare	Hides which of the two is weak; ignores true negatives
ROC-AUC	Classification	Chance a random positive is ranked above a random negative	Comparing ranking quality across all thresholds	Can look optimistic under heavy imbalance; says nothing about your chosen threshold
PR-AUC	Classification	Precision-recall trade-off across thresholds (average precision)	The positive class is rare	Its baseline equals the positive rate, so compare within one dataset
Log loss	Classification	Quality of predicted probabilities	The probabilities themselves drive decisions	Hard to interpret on its own

Why accuracy alone is not enough

Imagine 1,000 card transactions, 10 of them fraudulent. A model that always answers “not fraud” is 99.0% accurate and catches nothing. The model on the right is less accurate (95.8%) but finds 8 of the 10 frauds. Which one is better depends on the cost of a missed fraud versus a false alarm, which is why you choose the metric from the business goal, before modeling.

Always predicts “not fraud”

		PREDICTED: fraud	PREDICTED: not fraud
ACTUAL	fraud	**0** TP true positive	**10** FN missed fraud
	not fraud	**0** FP false alarm	**990** TN true negative

ACCURACY	PRECISION	RECALL	F1
99.0%	n/a	0.0%	0.00

99% accurate, yet it finds none of the 10 frauds. Precision is undefined because it never predicts fraud.

A useful fraud model

		PREDICTED: fraud	PREDICTED: not fraud
ACTUAL	fraud	**8** TP true positive	**2** FN missed fraud
	not fraud	**40** FP false alarm	**950** TN true negative

ACCURACY	PRECISION	RECALL	F1
95.8%	16.7%	80.0%	0.28

Less accurate, but it finds 8 of the 10 frauds, at the cost of 40 false alarms.

Model-selection checklist

- [ ] The metric reflects the real cost of each error type and was agreed before modeling.
- [ ] A baseline score is recorded, and every model is compared with it.
- [ ] All candidates use the same pipeline, splits and cross-validation folds.
- [ ] No test data influenced preprocessing, feature selection or tuning.
- [ ] Scores are reported with their spread across folds, not as a single number.
- [ ] The gap between training and validation scores has been checked.
- [ ] The decision threshold was chosen on validation data for the real objective.
- [ ] Errors were inspected by segment, so no group is failing silently.
- [ ] When scores are within the fold spread, the simpler model wins.
- [ ] The test set was used once, at the end, and the result is reported honestly.

PRACTICE TASK

Return to your Stage 4 comparison. Fix the metric that matches the real objective (for example recall, subject to precision of at least 50%, because a retention team can only call so many customers). Tune the two best models with cross-validation, choose a threshold on validation data, study the 20 worst errors, then evaluate the final model once on the test set. Summarize data, metric, result and limitations in a one-page model card.

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] You can diagnose overfitting or underfitting from training and validation scores.
- [ ] Given a business problem, you can choose a metric and explain why accuracy might mislead.
- [ ] You can read a confusion matrix and compute precision and recall by hand.

STAGE 07 APPLY & GROW

End-to-End Projects

07

Make real decisions on real data, and show them in a portfolio others can run.

Projects turn knowledge into skill. Start Project 1 during Stages 3 and 4, not after finishing the roadmap. Each project adds one new challenge. Do not copy a notebook: write down the decision at each step (which metric, which features, which model, and why) and let those decisions be the heart of your README.

Project progression



House Price Prediction
REGRESSION
New: Regression workflow, baselines, residuals

Customer Churn Prediction
CLASSIFICATION
New: Imbalance, precision and recall, thresholds

Customer Segmentation
CLUSTERING
New: Working without labels; judging usefulness

Term-Deposit Propensity Pipeline
ADVANCED TABULAR
New: Leakage control, reproducible pipeline, API-ready

P1 House Price Prediction

REGRESSION · START DURING STAGE 4

OBJECTIVE	Predict a home's sale value from its characteristics.
DATASET	Built-in: `sklearn.datasets.fetch_california_housing` (district-level values). Search term for house-level data: “Ames housing dataset”.
FORMULATION	Regression. Target: sale price (try log price if it is right-skewed). One row per home or district.
DATA PREPARATION	Missing values, encoding of location, features such as age at sale and total area, checks on skew.
BASELINE	Predict the median price for every home (`DummyRegressor`).
CANDIDATES	Linear regression, Ridge or Lasso, random forest, gradient boosting.
EVALUATION	5-fold cross-validation; MAE in currency units plus RMSE; residuals by price band.
DELIVERABLES	Notebook, results table against the baseline, top drivers by permutation importance, README.
PITFALLS	Features recorded after the sale; reporting log-scale errors as if they were currency; ignoring extreme properties.
EXTENSIONS	Prediction intervals with quantile loss; error analysis by neighborhood.

P2 Customer Churn Prediction

CLASSIFICATION · START AFTER STAGE 4

OBJECTIVE	Rank customers by risk of cancelling so a retention team can contact the right people.
DATASET	Search terms: “telco customer churn dataset” or “bank customer churn dataset”.
FORMULATION	Binary classification. Define the churn window (e.g. cancels within 30 days) and the prediction moment.
DATA PREPARATION	Fix numeric columns stored as text, encode contract and payment type, tenure features, stratified split.
BASELINE	Most-frequent-class dummy, then logistic regression.
CANDIDATES	Logistic regression, random forest, gradient boosting.
EVALUATION	Precision, recall and PR-AUC; choose the threshold from how many customers the team can contact each month.
DELIVERABLES	Model comparison, chosen threshold with rationale, list of top-risk customers with key drivers.
PITFALLS	Accuracy as the headline metric; features recorded after cancellation; oversampling before the split.
EXTENSIONS	Cost-based threshold; calibration curve; discuss who would actually respond to an offer.

P3 Customer Segmentation

CLUSTERING · START AFTER STAGE 5

OBJECTIVE	Group customers into segments that marketing could treat differently.
DATASET	Search term: “online retail transactions dataset UCI”. Aggregate transactions to one row per customer.
FORMULATION	Unsupervised. Recency, frequency and monetary features, optionally product mix.
DATA PREPARATION	Handle cancellations and returns explicitly, log-transform skewed spend, scale features.
BASELINE	Simple rule-based RFM tiers, to check that clustering adds something.
CANDIDATES	k-means, hierarchical clustering, DBSCAN for outliers; PCA for plots.
EVALUATION	Silhouette score, cluster sizes, stability across seeds and samples, and a distinct action per segment.
DELIVERABLES	Segment profile table, 2D PCA plot, a short persona and recommended action per segment.

P3 Customer Segmentation	CLUSTERING · START AFTER STAGE 5
PITFALLS	Unscaled features; a few very large customers defining a cluster; picking k from one metric alone.
EXTENSIONS	Compare with a Gaussian mixture model; track how customers move between segments.

P4 Term-Deposit Propensity Pipeline	ADVANCED TABULAR · START AFTER STAGE 6
OBJECTIVE	Predict which bank clients will subscribe to a term deposit, before a marketing call, with a pipeline anyone can retrain.
DATASET	Search term: "UCI bank marketing dataset". Read its documentation on which columns are known before a call.
FORMULATION	Binary classification at the moment before a call. Drop the call `duration` column: it is only known after the call.
DATA PREPARATION	Treat "unknown" categories deliberately; full `ColumnTransformer` pipeline; preprocessing tests.
BASELINE	Dummy classifier, then logistic regression.
CANDIDATES	Logistic regression, random forest, `HistGradientBoostingClassifier`, and one of XGBoost, LightGBM or CatBoost.
EVALUATION	Repeated stratified CV, PR-AUC and recall at a fixed call budget, tuning, threshold choice, error analysis, one final test run.
DELIVERABLES	`train.py` and `predict.py`, saved pipeline, config file, tests, model card, README, and an API (Stage 8).
PITFALLS	Leakage through `duration`; tuning on the test set; untracked experiments; code that only runs in a notebook.
EXTENSIONS	Experiment tracking with MLflow; a drift-monitoring sketch; model explanations with SHAP.

What a strong GitHub portfolio project contains

Reviewers look for evidence of decisions, not just a high score. Check each project against this list before you share it.

- ☐ **Clear README**
  What the project does and who would use it, in the first three lines.
- ☐ **Problem statement**
  Target, prediction moment and why the problem matters.
- ☐ **Data and preprocessing**
  Source, size, cleaning steps, features and anything you excluded.
- ☐ **Model selection rationale**
  Baseline, candidates, and why the final model won.
- ☐ **Appropriate metrics**
  The chosen metric, why it fits, and scores with spread.
- ☐ **Reproducible setup**
  `requirements.txt`, fixed seeds, data download or placement steps.
- ☐ **Results and limitations**
  What works, where it fails, and what you would do next.
- ☐ **Run instructions**
  Commands that take a new user from clone to prediction.

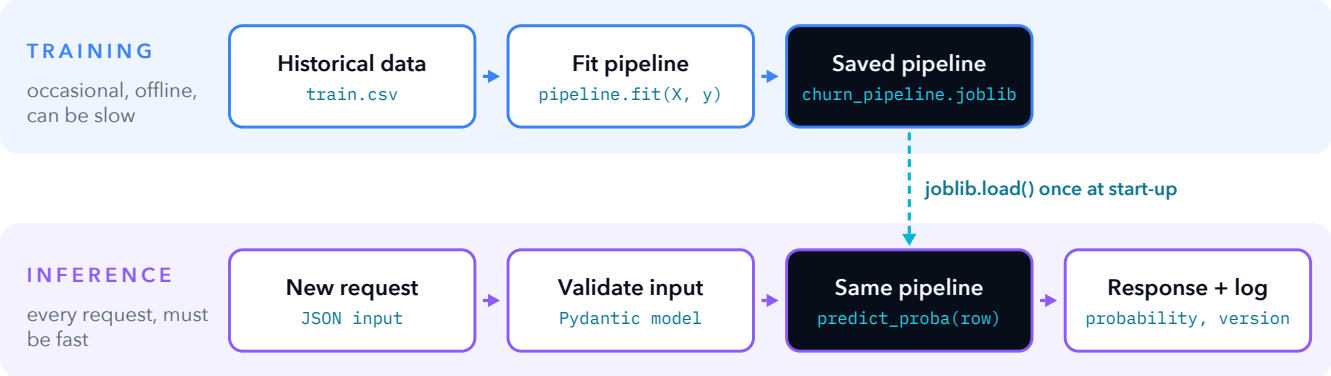
STAGE 08 APPLY & GROW

Deployment and Production Basics

Put a trained model behind a simple, reliable interface that someone else can use.

Training learns parameters from historical data; it runs occasionally and can be slow. **Inference** uses the trained model to predict for new inputs; it runs often, must be fast, and must apply exactly the same preprocessing as training. Your first deployment can be a small API running on one machine or a simple hosting service. Docker, Kubernetes, cloud architecture and full MLOps platforms are valuable later, but they are not prerequisites for deploying a first classical ML model.

Training versus inference



What to learn for a first deployment

TOPIC	WHAT TO LEARN
Saving and loading models	Save the whole fitted pipeline with `joblib.dump` and load it with `joblib.load`. Record library versions, because a saved model may not load under different versions. Only load model files from sources you trust: loading a pickle can execute code.
Reproducible inference	Save the pipeline, not just the estimator, so the same imputation, encoding and scaling run in production.
Prediction interface	Start with a `predict(record)` function, then wrap it in a web API. A small Streamlit or Gradio app is a friendly demo.
FastAPI	Define endpoints and request models with Pydantic; interactive documentation is generated at `/docs`.
Input validation	Check types, ranges and allowed categories; reject bad input with a clear error instead of returning a confident nonsense prediction.
Dependency management	Pin versions in `requirements.txt` and use the same versions for training and serving.
Logging and error handling	Log the model version, prediction, latency and errors. Be careful with personal data in logs.
Basic deployment	Run locally with `uvicorn`, then deploy to a hosting service. Add Docker when you need identical environments.

TOPIC	WHAT TO LEARN
Model and data versioning	Return a model version with every response, and record which data and code produced each model.
Monitoring and drift	Data drift: the input distribution changes. Concept drift: the relationship between inputs and target changes. Track input statistics, and performance once true labels arrive.
Limits after deployment	A model reflects the past. Performance decays, predictions can change behavior (feedback loops), and errors may fall unevenly on different groups.

Serving the Stage 3 pipeline with FastAPI

app.py — FASTAPI + PYDANTIC

```python
# app.py  -  run locally with:  uvicorn app:app --reload   (docs at /docs)
from typing import Literal

import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field

model = joblib.load("churn_pipeline.joblib")   # the whole fitted Pipeline
app = FastAPI(title="Churn prediction API")

class Customer(BaseModel):
    tenure_months: int = Field(ge=0, le=600)
    monthly_charges: float = Field(gt=0)
    contract: Literal["Month-to-month", "One year", "Two year"]
    payment_method: str

@app.post("/predict")
def predict(customer: Customer):
    row = pd.DataFrame([customer.model_dump()])
    proba = float(model.predict_proba(row)[0, 1])
    return {"churn_probability": round(proba, 3), "model_version": "1.0.0"}
```

Pydantic validates every request before the model sees it: a missing field, a negative tenure or an unknown contract type returns HTTP 422 with an explanation. An unseen payment method is still accepted, because the pipeline's encoder ignores unknown categories.

PRACTICE TASK

Train your churn pipeline, save it with `joblib`, and serve it with FastAPI. Test four requests: one valid, one with a missing field, one with an out-of-range value and one with an unseen category. Record each request and response in your README, and add one log line per prediction with the model version and latency.

READINESS CHECKPOINT

Move on when you can tick most of these without notes:

- [ ] You can explain the difference between training and inference, and why the full pipeline is saved.
- [ ] Your API returns predictions for valid input and a clear 422 error for invalid input.
- [ ] Another person can install and call your API by following only your README.

STAGE 09 APPLY & GROW

What to Learn Next

Choose one specialization based on the role you want, and go deep.

09

Classical ML gives you the habits every specialization relies on: clean data, honest evaluation and reproducible code. Each path below is a multi-month commitment. Choose one based on your target role and the projects that excite you; trying to master all three at once usually means finishing none.

Your classical ML foundation · Stages 1–8

Deep Learning

For: ML engineer (vision, speech, NLP), research engineer

- Neural network fundamentals: layers, activations, loss
- Backpropagation intuition: the chain rule, layer by layer
- PyTorch: tensors, autograd, modules, data loaders
- The training loop: forward pass, loss, backward pass, optimizer step, validation
- Introductions to computer vision (CNNs, transfer learning) and NLP (tokens, embeddings, transformers)

FIRST PROJECT

Fine-tune a pretrained image classifier on a small dataset you collect.

Generative AI and LLM Apps

For: AI engineer, applied LLM developer

- LLM fundamentals: tokens, context windows, prompting, known failure modes
- API integration: hosted models, structured output, cost and latency
- Embeddings and vector search
- Retrieval-Augmented Generation (RAG): chunking, retrieval, grounded answers
- Evaluation with test question sets; deployment and monitoring of quality and cost

FIRST PROJECT

A question-answering assistant over a document set, with its own evaluation set.

Production ML (MLOps)

For: MLOps engineer, ML platform engineer

- Experiment tracking, for example with MLflow
- Data and model versioning; model registries
- CI/CD concepts: automated tests and builds on every change
- Automated training pipelines with an orchestrator
- Monitoring, alerting, rollback and reliability; containers and cloud basics

FIRST PROJECT

Turn Project 4 into a pipeline that retrains, evaluates and registers a model automatically.

HOW TO CHOOSE

Choose by role. Want to build chat assistants and document search? Start with Generative AI. Want to train vision or speech models? Start with Deep Learning. Enjoy automation, reliability and infrastructure? Start with Production ML. Whichever you pick, keep using Stages 3 and 6: data quality and honest evaluation matter on every path.

LEARNING PLAN

An Illustrative 12-Week Plan

READ THIS FIRST

Illustrative schedule only. It assumes about 12 to 15 focused hours a week and some prior programming comfort. Many learners take longer, especially when starting Python from zero or studying part-time, and that is normal. Adjust the pace, but keep the pattern: a project running alongside the theory in most weeks.

Schedule at a glance



Blue: foundations. Cyan: core machine learning. Violet: projects, deployment and revision. P1 to P4 mark when each project is completed.

Week by week

WEEK	FOCUS	OUTPUT BY THE END OF THE WEEK
01	Core Python: syntax, functions, data structures, files, virtual environments	Ten small scripts
02	NumPy, pandas, Matplotlib, Seaborn; descriptive statistics	Stage 1 practice notebook
03	Probability, distributions, linear algebra essentials; start EDA	Stage 2 practice notebook
04	Cleaning, encoding, splits, leakage, pipelines; gradients and loss	Project 1: data prep and baseline
05	Linear, Ridge and Lasso regression; MAE, RMSE, cross-validation	Project 1 published
06	Logistic regression, kNN, Naive Bayes, decision trees	Stage 4 comparison notebook
07	Random forests, gradient boosting, SVMs; classification metrics	Project 2: baseline and candidates
08	Thresholds, tuning, error analysis, feature importance	Project 2 published, model card
09	k-means, hierarchical clustering, DBSCAN, PCA	Project 3 published
10	Project 4: pipeline, model comparison, tests	Project 4 modeling complete
11	Saving models, FastAPI, validation, logging	Project 4 served through an API
12	Revise weak areas, polish READMEs, choose a next path	Portfolio of four projects

TRACK YOUR PROGRESS

Progress Tracker

Tick each milestone and write the date. Seeing your pace helps you adjust the plan honestly.

	MILESTONE	STAGE	DATE COMPLETED
☐	Practice notebook: load, clean, summarize and chart a dataset	Stage 1	____ / ____ / ______
☐	Statistics notebook and gradient descent from scratch in NumPy	Stage 2	____ / ____ / ______
☐	Preprocessing pipeline with a written decision log	Stage 3	____ / ____ / ______
☐	Project 1: House price prediction on GitHub	Stage 7	____ / ____ / ______
☐	Model comparison notebook with a baseline and three models	Stage 4	____ / ____ / ______
☐	Project 2: Customer churn prediction on GitHub	Stage 7	____ / ____ / ______
☐	Customer segmentation notebook with cluster profiles	Stage 5	____ / ____ / ______
☐	Project 3: Customer segmentation on GitHub	Stage 7	____ / ____ / ______
☐	Model card for your best classification model	Stage 6	____ / ____ / ______
☐	Project 4: Reproducible tabular pipeline on GitHub	Stage 7	____ / ____ / ______
☐	API serving a model, tested with valid and invalid requests	Stage 8	____ / ____ / ______
☐	Specialization chosen, first project scoped	Stage 9	____ / ____ / ______

Final readiness checklist

When you can honestly tick these, you have strong foundations and are ready to specialize or apply for junior roles.

- ☐ I can take a raw tabular dataset to a cross-validated model inside one reproducible pipeline.
- ☐ I can choose a metric that matches the problem and explain when accuracy misleads.
- ☐ I can explain how my main models work and why I chose the final one.
- ☐ I can spot and remove data leakage in my own and other people's work.
- ☐ I can explain results and limitations to a non-technical person.
- ☐ I have at least three projects on GitHub with READMEs that someone else can follow.
- ☐ I have served a model through an API that validates its input.
- ☐ I know which specialization I will pursue next, and why.

Ticking every box shows solid foundations and a portfolio to talk about. It is not a guarantee of a job or of mastery: hiring depends on many factors, and learning continues long after this roadmap.

LEARNING RESOURCES

Learning Resources

Pick **one** primary resource per stage, work through it with your own code, and use the documentation to look things up. All links were checked when this edition was produced. Datasets are given as search terms elsewhere in this roadmap, because dataset pages move; read a dataset's documentation and license before using it publicly.

Primary learning resources (choose one per stage)

RESOURCE	USEFUL FOR	STAGE
The Python Tutorial · Python Software Foundation docs.python.org/3/tutorial/	The official introduction to the language, including virtual environments.	1
Kaggle Learn · Kaggle kaggle.com/learn	Short hands-on courses that run in the browser.	1, 3, 4
Statistics and probability · Khan Academy khanacademy.org/math/statistics-probability	Lessons and exercises for the Stage 2 statistics.	2
An Introduction to Statistical Learning (with Python) · James, Witten, Hastie, Tibshirani, Taylor statlearning.com/	The classical algorithms explained clearly; the official site offers a PDF download.	2, 4–6
Machine Learning Crash Course · Google for Developers developers.google.com/machine-learning/crash-course	Concise modules on core ML concepts and evaluation.	4, 6
Mathematics for Machine Learning · Deisenroth, Faisal, Ong mml-book.github.io/	The Deepen later math; the authors keep a PDF freely available.	2
FastAPI Tutorial - User Guide · FastAPI fastapi.tiangolo.com/tutorial/	Building and validating APIs, step by step.	8
Learn the Basics · PyTorch Tutorials docs.pytorch.org/tutorials/beginner/basics/intro.html	Tensors, autograd and the training loop.	9

Reference documentation (look things up as you build)

DOCUMENTATION PAGE	STAGES
NumPy: the absolute basics for beginners · NumPy docs numpy.org/doc/stable/user/absolute_beginners.html	1
pandas: getting started tutorials · pandas docs pandas.pydata.org/docs/getting_started/intro_tutorials/index.html	1, 3
Matplotlib: quick start guide · Matplotlib docs matplotlib.org/stable/users/explain/quick_start.html	1
Seaborn: user guide and tutorial · Seaborn docs seaborn.pydata.org/tutorial.html	1, 3
scikit-learn User Guide · scikit-learn docs scikit-learn.org/stable/user_guide.html	3 to 6
scikit-learn: common pitfalls and recommended practices · scikit-learn docs scikit-learn.org/stable/common_pitfalls.html	3, 6
scikit-learn: metrics and scoring · scikit-learn docs scikit-learn.org/stable/modules/model_evaluation.html	6
scikit-learn: tuning the decision threshold · scikit-learn docs scikit-learn.org/stable/modules/classification_threshold.html	6
scikit-learn: model persistence · scikit-learn docs scikit-learn.org/stable/model_persistence.html	8

SkillPilot

Learn in sequence.
Practice early. Build real projects.

The best time to start the Stage 1 practice task is today.

01 **Start small, today**
Open a notebook, load one dataset, and answer one question with a chart.

02 **Build in public**
Push every project to GitHub with a README, even when it is not perfect.

03 **Come back to this roadmap**
Tick the checkboxes, revisit the checkpoints, and adjust your plan.

More free SkillPilot resources
vikasthamke.github.io/skillpilot-resources

SkillPilot on YouTube
youtube.com/@TheSkillPilot

NAVIGATE YOUR SKILLS. BUILD YOUR FUTURE.